SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

삼성청년 SW·AI아카데미

AI 코어 PJT

Confidential

<알림>

본 강의는 삼성청년SW·AI아카데미의 컨텐츠로
보안서약서에 의거하여
강의 내용을 어떠한 사유로도 임의로
복사, 촬영, 녹음, 복제, 보관, 전송하거나
허가 받지 않은 저장매체를 이용한 보관,
제3자에게 누설, 공개,
또는 사용하는 등의 행위를 금합니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Day1. PoC 제작 & Object Detection 파인튜닝

Confidential

챕터의 포인트

- 멀티모달 인식이 필요한 이유 + 환경 세팅
- Object Detection 모델 선정 및 테스트
- VLM 모델 선정 및 테스트
- 데모 PoC 통합 (YOLO + VLM)
- Object Detection 모델 Finetuning

Confidential

2회차 목표 : 멀티모달 인식 통합과 의도 파악 AI Workflow

- **오늘은 '모델을 고르고 검증하는 날' - 완벽한 정확도보다 기록이 목표**
- **무엇이 되고 무엇이 안 되는지를 숫자로 남긴다**

차시	주제	핵심 수행 과제	산출물
1일차 (오늘)	PoC 제작 + Object Detection 파인튜닝	VLM / Object Detection 모델 선정 · 테스트, 데모 PoC, YOLO Finetuning	동작하는 PoC 코드 + 파인튜닝 모델
2일차	멀티모달 인식 모델 연동	YOLO · VLM · 제스처 결과를 Text 로 → 웹 대시보드 서버 → LLM 통합	웹 대시보드 서버 (인식 + LLM 답변)
3일차	AI Workflow 구현	의도 파악 + 도구 호출(tool call) 조건 분기, 시뮬레이터 연동	대시보드 + 시뮬레이터 동작 데모

Confidential

학습 목표

- **① 고른다 - 근거를 들어 모델을 선택한다**
  - 정확도 · 속도 · 메모리 · 라이선스로 비교하고, 고른 이유를 네 줄로 말할 수 있으면 통과
- **② 돌린다 - 고른 모델을 내 노트북에서 직접 실행한다**
  - 같은 이미지로 출력과 추론 시간을 재서, 내 PC의 숫자를 손에 쥐면 통과
- **③ 묶는다 - 두 모델을 하나의 서버로 통합한다**
  - 이미지 1장에 객체 요약과 장면 설명이 함께 돌아오면 통과
- **④ 고친다 - 우리 환경에 맞게 모델을 다시 학습시킨다**
  - 라벨링 → 파인튜닝 → 전/후 비교까지 한 바퀴를 돌면 통과

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

멀티모달 인식과 환경 세팅

Confidential

같은 사진 한 장, 질문 하나 - "앞에 뭐가 있고 피해서 갈 수 있을까?"

- **셋을 경쟁시키는 것이 아니라 역할을 나눠 연결한다 - AI Workflow 설계의 출발점**

모델	잘하는 것	못하는 것
Object Detection (YOLO)	객체의 종류 · 개수 · 위치를 좌표와 확신도로 정확히 특정	장면의 맥락 · 관계 · 상황 설명 불가 (학습한 클래스만 인식)
VLM (Qwen2.5-VL)	장면 전체를 자연어로 서술, 학습하지 않은 상황도 유연하게 표현	개수 · 위치가 부정확하고 환각(hallucination) 발생
LLM (Qwen3)	텍스트 근거를 종합해 의도 파악 · 판단 · 도구 호출	이미지를 직접 볼 수 없음 (텍스트 입력만 가능)

Confidential

병렬 인식 → 텍스트 통합 → 단일 LLM 판단 → Action

- **① 입력 - 카메라 이미지 + 사용자 질문**
  - 이미지 한 장과 자연어 질문이 같은 요청으로 들어온다
- **② 인식 (여러 모델을 병렬로) - 모두 출력은 '텍스트'**
  - YOLO11n : 이미지 → "person 4개, bus 1개" ← 오늘 만든다
  - VLM (Qwen2.5-VL-3B) : 이미지 → 장면 설명 문장 ← 오늘 만든다
  - 손 제스처 : 카메라 → "가리키는 방향: 위" ← 2일차 (음성은 3회차)
- **인식 단계의 핵심 규칙**
  - 출력 형식이 제각각이면 통합이 불가능하다 → 전부 텍스트로 통일한다
  - 개수는 YOLO가, 맥락은 VLM이 - 잘하는 것만 시킨다

Confidential

텍스트로 모인 근거를 LLM 하나가 판단한다

- **③ 통합 - LLM (Qwen3-4B)**
  - Text 묶음 → 상황 이해 · 의도 파악 ← 2일차
  - 이미지를 직접 보지 않고, 인식 모델이 만든 텍스트만 읽는다
- **④ 행동 - 도구 호출 (tool call)**
  - 의도 → 시뮬레이터 동작으로 변환 ← 3일차
  - 조건 분기로 '무엇을 호출할지'를 LLM이 결정한다
- **오늘(1일차)의 담당 범위**
  - 가운데 인식 블록 2개(YOLO · VLM)만 만든다
  - 범위를 좁게 잡는 것이 PoC 성공의 첫 번째 조건이다

Confidential

가져가는 것은 세 가지 - 2일차 서버와 3회차 이식의 출발점

- **① 동작하는 PoC 서버 - 엔드포인트 2개**
  - 이미지 1장 → 객체 요약 문장 / 장면 설명 문장
  - 2일차 웹 대시보드 서버가 같은 '이미지 in → 텍스트 out' 구조를 이어받는다
- **② 내 PC의 성능 기록 - measurement.csv**
  - 추론 시간 · 메모리 · 검출 결과를 숫자로 남긴 표 - 모델 선정의 근거
- **③ 우리 데이터로 학습한 가중치 - best.pt**
  - 파인튜닝 전/후 비교표가 함께 남는다. 3회차 라즈베리파이 이식 때 다시 쓴다

Confidential

터미널에 한 줄씩 그대로 입력한다 (약 3분)

- **프롬프트 앞에 (.venv) 가 보이면 활성화 성공 · llama-cpp-python 은 버전을 고정한다**

```
# 0) 파이썬 버전 확인 (3.12 권장)
python --version

# 1) 프로젝트 폴더로 이동 — 배포 폴더(README.md · source)는
#    반드시 C:\work\core_pjt 에 둔다. 다른 곳이면 먼저 옮긴다
cd C:\work\core_pjt

# 2) 가상환경 생성 후 활성화
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # macOS / Linux

# 3) 패키지 설치 — 한 줄씩 그대로 (약 3분)
python -m pip install --upgrade pip
pip install fastapi==0.141.1 uvicorn==0.54.0 pillow==12.3.0 httpx==0.28.1 psutil==7.2.2
pip install ultralytics==8.4.164 torch==2.14.0 torchvision==0.29.0
pip install llama-cpp-python==0.3.34 --extra-index-url https://abetlen.github.io/llama-cpp-python/whl/cpu
```

전체 소스 : 설치_명령어.sh
https://gist.github.com/maketrust7/cd336451907fa0e9afceaccfdc4048d1

Confidential

가중치는 pip 로 설치되지 않는다 - hf download 로 받는다

- **YOLO 는 자동, VLM 은 수동 · 수업 전에 받아 두면 20분을 번다 (3.1 GB, 15 MB/s 기준 약 4분)**

```
# YOLO11n (5.4MB) — page23 실습을 처음 돌릴 때 자동 다운로드

# hf 명령 설치 (HuggingFace 다운로드 도구) — 1번과 같은 폴더에서
cd C:\work\core_pjt
pip install huggingface_hub==2.0.0

# VLM 본체(1.9GB) + mmproj(1.3GB) — 둘 다 있어야 이미지 입력이 된다
hf download ggml-org/Qwen2.5-VL-3B-Instruct-GGUF Qwen2.5-VL-3B-Instruct-Q4_K_M.gguf --local-dir models
hf download ggml-org/Qwen2.5-VL-3B-Instruct-GGUF mmproj-Qwen2.5-VL-3B-Instruct-f16.gguf --local-dir models

# LLM(2.4GB) — 2일차부터 쓴다. 오늘은 없어도 된다
hf download Qwen/Qwen3-4B-GGUF Qwen3-4B-Q4_K_M.gguf --local-dir models
```

models₩ 에 놓을 파일	크기	언제 쓰나	없으면
yolo11n.pt (프로젝트 루트)	5.4 MB	Object Detection · 파인튜닝	자동 다운로드됨
Qwen2.5-VL-3B-Instruct-Q4_K_M.gguf	1.9 GB	VLM · PoC 통합	VLM 실행 불가
mmproj-Qwen2.5-VL-3B-Instruct-f16.gguf	1.3 GB	VLM · PoC 통합	이미지 입력이 동작 안 함
Qwen3-4B-Q4_K_M.gguf	2.4 GB	2일차	오늘은 없어도 된다

전체 소스 : 설치_명령어.sh
https://gist.github.com/maketrust7/cd336451907fa0e9afceaccfdc4048d1

Confidential

코드는 사전 배포된다 - 열어서 읽고 고치는 방식

- **① File → Open Folder → C:₩work₩core_pjt**
- **② 확장 설치 : Python (Microsoft)**
- **③ Ctrl+Shift+P → Select Interpreter → .venv 선택**
- **④ 통합 터미널 Ctrl+` - PoC 통합은 터미널 2개**
- **⑤ 실습 폴더로 이동 후 실행 (cd page15_환경체크)**

③에서 .venv 가 목록에 없으면

열린 폴더에 .venv 가 없는 것이다 (VSCode 는 연 폴더 안만 찾는다)

· 제목 표시줄이 core_pjt 인지 확인 → 아니면 ①부터 다시

· 그래도 없으면 Enter interpreter path →

C:₩work₩core_pjt₩.venv₩Scripts₩python.exe

```
C:\work\core_pjt\
 ├── yolo11n.pt                    YOLO 가중치
 ├── models\                       VLM 가중치 2개
 └── source\2회차_day1\            ← 실습 코드
     ├── README.md                 실습 순서 안내
     ├── samples\                  공용 입력 이미지
     ├── dataset\  runs\           공용 산출물 (파인튜닝이 생성)
     ├── page15_환경체크\           실습 한 건 = 폴더 한 개
     │     ├── README.md           무엇을 / 어떻게 / 기대 결과
     │     ├── 환경체크_설치확인.py
     │     └── 실행결과.txt          실제 출력
     ├── page25_YOLO_첫추론\
     └── ... (쪽번호 = 이 강의자료 슬라이드 번호)
```

Confidential

설치가 끝났는지 5분 안에 확인한다

- **① 패키지 ② 추론 디바이스 ③ 가중치 파일 순으로 점검**
- **실행 : python 환경체크_설치확인.py**

```python
REQUIRED_PACKAGES = [
    ("fastapi", "PoC 서버"),
    ("uvicorn", "PoC 서버 실행"),
    ("httpx", "PoC 클라이언트"),
    ("psutil", "성능 측정 (메모리)"),
    ("PIL", "이미지 처리 (pillow)"),
    ("ultralytics", "YOLO Object Detection"),
    ("torch", "YOLO 백엔드"),
    ("llama_cpp", "VLM GGUF 추론 (llama-cpp-python)"),
]

# ...

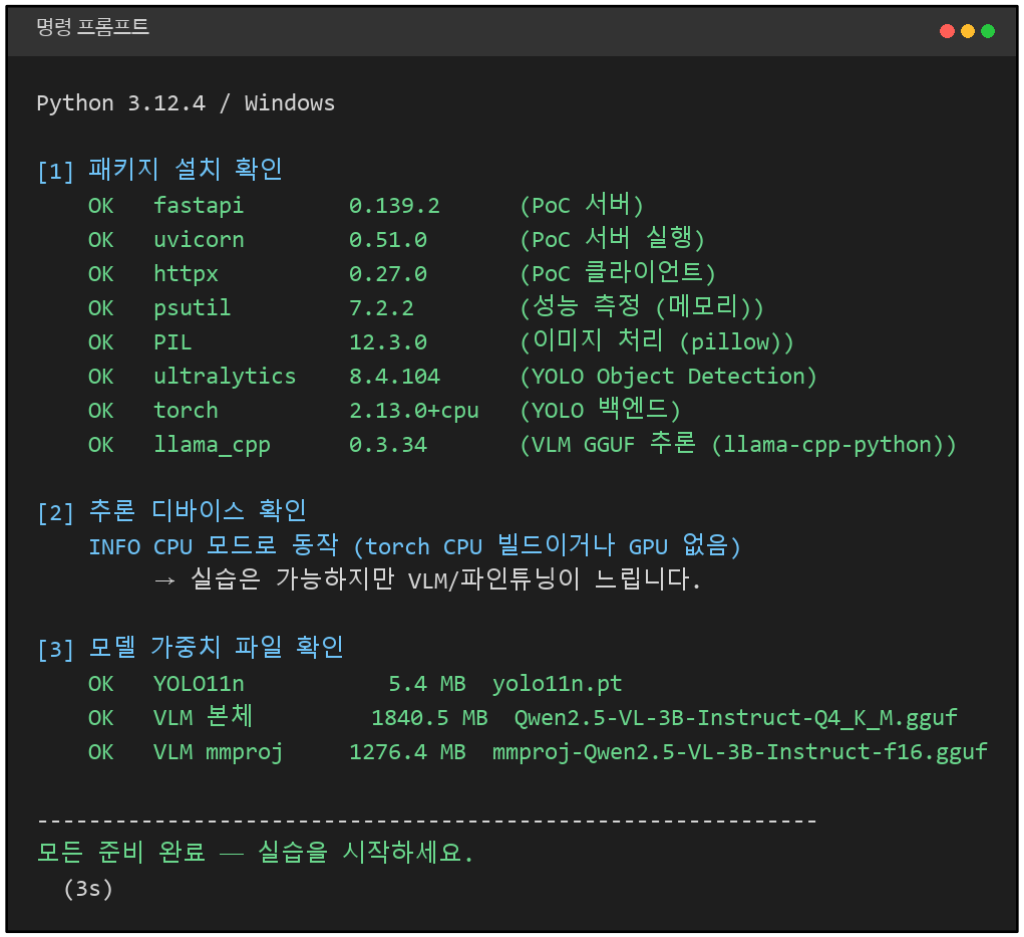
    for path, label, hint, auto in targets:
        if path.exists():
            size_mb = path.stat().st_size / 1024 / 1024
            print(f"    OK   {label:<12} {size_mb:8.1f} MB  {path.name}")
        elif auto:                                      # 정상 — 실패로 세지 않는다
            print(f"    AUTO {label:<12} {'':>8}      {path.name}")
            print(f"         → {hint}")
        else:
            ok = False
            print(f"    MISS {label:<12} {'':>8}      {path}")
            print(f"         → {hint}")
    return ok
```

전체 소스 : page15_환경체크/환경체크_설치확인.py
https://gist.github.com/maketrust7/cae90989b4ac22b9d5bd8e725fd2df7d

Confidential

MISS 가 하나라도 보이면 실습을 시작하지 않는다

- **이 단계에서 해결하는 것이 나중에 실습이 밀리는 것 보다 빠르다**
- **실측 : 패키지 8개 · 가중치 3개 모두 OK (CPU 모드)**
- **yolo11n.pt 가 없으면 AUTO - 다음 실습에서 자동으로 받는다 (정상)**



Object Detection 선정 · 테스트

Confidential

Object Detection 의 출력은 항상 세 가지가 한 묶음이다

- **bbox - '어디에' 있는가**
  - (x1, y1, x2, y2) 픽셀 좌표. 이미지 크기가 바뀌면 같이 바뀐다
- **class - '무엇이' 있는가**
  - 학습된 클래스 인덱스를 이름으로 매핑한다 (COCO 기준 80종)
  - 학습하지 않은 물체는 아예 후보에 오르지 않는다
- **confidence - '얼마나 확실한가'**
  - 0~1 점수. 이 값을 어디서 자르느냐가 결과를 가장 크게 바꾼다
- **이 세 가지를 LLM이 읽을 한 문장으로 줄이는 것이 목표**

Confidential

conf / iou 두 개의 임계값이 결과를 좌우한다

- **conf - 이 점수 미만의 검출은 버린다**
  - 올리면 : 검출 수 ↓, 놓침(누락) ↑ - 장애물을 못 보게 된다
  - 내리면 : 검출 수 ↑, 오탐(False Positive) ↑ - 헛것을 보게 된다
  - 실측 : conf 0.10 에서 오탐 skateboard 가 하나 더 잡혔다
- **iou - NMS(중복 박스 제거)의 기준**
  - 올리면 : 겹친 박스를 더 많이 남긴다
  - 내리면 : 겹친 객체를 하나로 합쳐버린다
- **두 값은 '학습'이 아니라 '운영'에서 정하는 값 - 현장에서 튜닝한다**
  - 그래서 파인튜닝 평가 결과를 읽을 때도 conf 값을 같이 봐야 한다 (도전 4)

Confidential

무엇으로 '좋은 모델'을 판단하는가

- **mAP50 / mAP50-95 - 점수 순으로 줄 세웠을 때 정답이 얼마나 앞에 오는가**
  - IoU 0.5 기준 / 0.5~0.95 평균. 순위를 보는 지표라 '확신의 크기' 는 안 본다 (도전 4)
- **precision - 검출한 것 중 맞은 비율. 낮으면 헛것을 본다 (괜히 멈춤)**
- **recall - 실제 객체 중 찾아낸 비율. 낮으면 놓친다 (부딪힘)**
  - 둘은 conf 임계값에 따라 함께 움직인다 - 한쪽을 올리면 다른 쪽이 내려간다
- **FPS / latency - 3회차 온디바이스 이식 가능 여부를 결정하는 실전 지표**

Confidential

선정 기준 4가지 : 정확도 · 속도 · 메모리 · 라이선스

- **3회차에 라즈베리파이5로 이식 → 속도 · 메모리의 가중치를 높게 둔다**
- **"가장 정확한 모델"이 아니라 "조건을 만족하는 가장 가벼운 모델"**
- **라이선스 : 수업 · 연구는 문제없음. AGPL 은 서비스로 배포하면 소스 공개 의무 - 상용 전 확인**

모델	파라미터	정확도(COCO mAP)	CPU 속도	라이선스	특징
YOLO11n	약 2.6M	낮음~중간	가장 빠름	AGPL-3.0	온디바이스 이식 용이, 파인튜닝 간단
YOLO11s / m	9.4M / 20M	중간~높음	느려짐	AGPL-3.0	정확도 우선일 때, PC 전용
RT-DETR	약 20M+	높음	느림	Apache-2.0	NMS 불필요, 학습 비용 큼

Confidential

모델 선정은 취향이 아니라 근거다 - 네 줄로 쓸 수 있어야 한다

- **① 3회차 온디바이스 이식이 전제다 - 속도 · 메모리 우선**
  - 확인 방법 : 직접 측정한 추론 시간 (실측 44.7 ms / 22.4 fps, 실습 2-2)
- **② 필요한 정보는 '객체 종류와 개수' 뿐이다**
  - 박스 위치 정밀도(mAP50-95)는 희생해도 무방하다
- **③ ultralytics 한 줄 API 로 파인튜닝까지 이어진다**
- **④ 가중치가 약 5.4 MB 로 작다 - 배포 · 이식 부담이 거의 없다**

Confidential

동시 요청에도 모델은 딱 1개만 올라가야 한다

- **① 락 없이 빠르게 검사 ② 초기화 구간만 잠금 ③ 락 획득 후 재검사**
- **③이 없으면 동시 요청 시 모델이 두 번 올라가 메모리가 2배가 된다**

```python
def get_yolo_model():
    global _yolo_model, _load_count
    if _yolo_model is None:                # ① 락 없이 빠르게 검사
        with _yolo_init_lock:              # ② 초기화 구간만 잠근다
            if _yolo_model is None:        # ③ 락 획득 후 한 번 더 검사
                from ultralytics import YOLO

                model = YOLO(str(YOLO_MODEL_PATH))
                model.to(_resolve_device())
                _load_count += 1
                _yolo_model = model
    return _yolo_model

# ...

    with ThreadPoolExecutor(max_workers=8) as pool:
        models = list(pool.map(lambda _: get_yolo_model(), range(8)))
```

전체 소스 : page23_모델로딩_지연과락/YOLO모델_지연로딩과락.py
https://gist.github.com/maketrust7/30ee20d5c81a7934705d925a20856b76

Confidential

스레드 8개가 같은 인스턴스 1개를 공유한다

- **실측 : 반환 객체 8개, 서로 다른 인스턴스 1개, 실제 로드 1회**
- **락을 빼면 _load_count 가 2 이상으로 올라간다**

명령 프롬프트

```
가중치 경로 : C:\work\core_pjt\yolo11n.pt
디바이스    : cpu

스레드 8개에서 동시에 get_yolo_model() 호출...
  반환된 객체 수         : 8
  서로 다른 모델 인스턴스: 1
  실제 로드 횟수         : 1
  소요 시간              : 0.20s

[PASS] 8개 스레드가 같은 모델 인스턴스 1개를 공유합니다.

(참고) _yolo_init_lock 을 제거하면 _load_count 가 2 이상으로 올라가며,
       VLM처럼 2GB짜리 모델이라면 메모리가 그만큼 배로 늘어납니다.
```

Confidential

이미지 1장을 넣고 박스 · 클래스 · 확신도를 받는다

- **실행 : python YOLO_첫추론.py --sweep**
- **--sweep 은 conf 0.1 / 0.25 / 0.6 을 한 번에 비교한다**

```python
def load_model():
    from ultralytics import YOLO

    return YOLO(str(YOLO_MODEL_PATH))

# ...

def detect(model, image: Image.Image, conf: float):
    t0 = time.perf_counter()
    results = model.predict(image, conf=conf, verbose=False)
    elapsed_ms = (time.perf_counter() - t0) * 1000
    return results, elapsed_ms
```

전체 소스 : page25_YOLO_첫추론/YOLO_첫추론.py
https://gist.github.com/maketrust7/dbae1f8553ae6352c8b88f9854a79630

Confidential

conf 를 낮추면 검출 수가 늘지만 오탐도 함께 는다

- **실측 : conf 0.25에서 bus 1 / person 4**
- **conf 0.10 에서는 오탐 skateboard 가 하나 더 잡힌다**
- **0.10 의 1668 ms 는 첫 추론(워밍업) 비용 - conf 와 무관**

명령 프롬프트

```
입력 이미지 : sample.jpg  (810x1080)
모델 로딩   : 1.48s

  conf |   검출 수 |    추론(ms) | 검출 결과
------------------------------------------------------------------------
  0.10 |      6 |      1668 | bus, person, person, person, person, skateboard
  0.25 |      5 |        95 | bus, person, person, person, person
  0.60 |      5 |        63 | bus, person, person, person, person

conf 를 낮추면 검출 수가 늘지만 오탐(False Positive)도 함께 늘어난다.
```

Confidential

LLM 은 좌표를 이해하지 못한다

- **지금 손에 있는 출력 - 기계가 쓰기 좋은 형태**
  - person conf=0.89 xyxy=[221.5, 405.8, 344.9, 857.6]
  - 클래스 · 좌표 · 확신도. 정확하지만 LLM 에게는 읽을 수 없는 잡음이다
- **LLM 에 필요한 출력 - 사람이 쓰는 한 문장**
  - "person 4개, bus 1개"
  - 좌표를 버리고 '무엇이 몇 개'만 남긴다
- **이 변환은 2일차에 만든다**
  - 출력 포맷을 정하는 일은 LLM 입력 설계의 일부다
  - VLM 설명문과 함께 하나의 입력 템플릿으로 묶어야 의미가 있다

Confidential

숫자를 남겨야 선정 근거가 된다

- **실행 : python YOLO_성능측정_기록표.py --runs 10**
- **결과는 measurement.csv 로 저장 (중간 결과)**

```python
def benchmark(weight_name: str, image: Image.Image, runs: int, conf: float) -> dict:
    from ultralytics import YOLO

    mem_before = current_memory_mb()

    t0 = time.perf_counter()
    model = YOLO(resolve_weight(weight_name))
    load_s = time.perf_counter() - t0

    # 첫 추론에는 초기화 비용이 섞이므로 워밍업 1회는 평균에서 제외한다.
    model.predict(image, conf=conf, verbose=False)

    durations = []
    for _ in range(runs):
        t0 = time.perf_counter()
        results = model.predict(image, conf=conf, verbose=False)
        durations.append((time.perf_counter() - t0) * 1000)

    mem_after = current_memory_mb()
    names = [model.names[int(b.cls)] for b in results[0].boxes]

    return {
        "model": weight_name,
        "load_s": round(load_s, 2),
        "infer_ms_avg": round(sum(durations) / len(durations), 1),
        "fps": round(1000 / (sum(durations) / len(durations)), 1),
        "detected": len(names),
        "mem_delta_mb": round(mem_after - mem_before, 1),
    }
```

전체 소스 : page28_YOLO_성능측정/YOLO_성능측정_기록표.py
https://gist.github.com/maketrust7/332c6d93f490d7be75707595b32f9496

Confidential

PC마다 값이 다르다 - 내 PC의 숫자를 남긴다

- **실측 : 44.7 ms 평균 / 22.4 fps / 메모리 +169 MB**
- **모델 선정 근거로 measurement.csv 를 남긴다**

명령 프롬프트

```
측정 중: yolo11n.pt (10회) ...

model |       load_s | infer_ms_avg | infer_ms_min | infer_ms_max |         fps |     detected
           |      objects | mem_delta_mb
-------------------------------------------------------------------------------------------------
    ---------------------------------------
yolo11n.pt |         0.04 |         44.7 |         43.7 |         46.5 |         22.4 |
      5 |   bus person |        169.2

CSV 저장: C:\work\vlm9\source\2회차_day1\page28_YOLO_성능측정\measurement.csv
이 파일은 중간 결과입니다. 3일차까지 보관하세요.
  (5s)
```

Confidential

실습 중 막히면 여기부터 확인한다

- **첫 호출은 느릴 수 있다**
- **"원하는 객체가 안 잡힘"이 파인튜닝으로 가는 주된 이유다**

증상	원인	해결
첫 호출만 느림	모델 지연 로딩 + 첫 추론 초기화	정상. 두 번째부터 빨라진다 (PoC 통합에서 확인)
GPU가 있는데 CPU로 동작	torch가 CPU 빌드로 설치됨	torch.cuda.is_available() 확인 후 cu128 빌드 재설치
원하는 객체가 안 잡힘	COCO 80개 클래스에 없는 객체	→ 파인튜닝으로 해결
파일 경로 오류	실행 디렉터리(cwd) 기준 상대경로	Path(__file__).resolve() 기준 절대경로 사용

Confidential

실습 2-1 --sweep 결과를 보고 장애물 판단용 conf 를 하나 고르시오.

실측 (sample.jpg)

conf 0.10 → 6개 : bus, person ×4, skateboard

conf 0.25 → 5개 : bus, person ×4

conf 0.60 → 5개 : bus, person ×4

Q. 0.10 / 0.25 / 0.60 중 무엇? 근거 두 줄

힌트

· 놓침과 오탐 중 로봇에게 더 위험한 것은?

· skateboard 는 실제로 있는가?

· 0.25 와 0.60 은 이 사진에서 같다

  - 다른 사진에서도 같을까?

도전 1 해설 - 오탐을 지우는 가장 낮은 값

Confidential

정답 : 0.25 - 놓침이 더 위험하므로 필요 이상 올리지 않는다

- **0.10 : 없는 skateboard 를 본다 → 오탐. 로봇이 괜히 피하거나 멈춘다**
- **0.25 : 오탐은 사라지고 사람 4 · 버스 1 은 모두 남는 가장 낮은 값**
- **0.60 : 이 사진에서는 같지만 가장 낮은 person 점수가 0.62 - 조금만 멀거나 가려져도 놓친다**
- **놓침(장애물을 못 봄)은 부딪힘으로 이어진다 - 임계값은 운영 목적에 맞춰 정한다**

실습 2-1 --sweep (실측)

```
입력 이미지 : sample.jpg  (810x1080)
모델 로딩   : 1.48s

  conf |   검출 수 |    추론(ms) | 검출 결과
------------------------------------------------------------------------
  0.10 |      6 |      1668 | bus, person, person, person, person, skateboard
  0.25 |      5 |        95 | bus, person, person, person, person
  0.60 |      5 |        63 | bus, person, person, person, person

conf 를 낮추면 검출 수가 늘지만 오탐(False Positive)도 함께 늘어난다.
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

VLM 모델 선정 및 테스트

Confidential

이미지 + 텍스트를 함께 입력받아 텍스트를 생성하는 모델

- **구조 : Vision Encoder → Projector(mmproj) → LLM Decoder**
- **그래서 VLM 실행에는 본체 가중치 + mmproj 두 파일이 모두 필요하다**
- **VLM에게 개수를 묻지 마세요 - 개수는 YOLO, 상황 설명은 VLM**

	Object Detection	VLM
출력 형태	구조화된 데이터(박스 · 클래스)	자유 형식 자연어
인식 범위	학습한 클래스만	학습하지 않은 상황도 서술 가능
개수 · 위치 정확도	높음	낮음 (환각 발생)
맥락 이해	없음	높음 ('사람이 문 앞에 서 있다')

Confidential

선정 : Qwen2.5-VL-3B-Instruct (GGUF Q4_K_M)

- **한국어 프롬프트에 한국어로 응답 - 사용자 질문과 같은 언어 유지**
- **4bit 양자화 시 약 1.8GB → 일반 노트북 CPU에서도 실행 가능**
- **라이선스 : Qwen2.5-VL-3B 는 연구용(비상업) - 수업은 문제없음, 상용 전 교체 검토**

모델	크기	한국어	경량화 지원	라이선스	비고
Qwen2.5-VL-3B-Instruct	3B	양호	GGUF 다수	Qwen Research	한국어 품질 대비 가벼움
SmolVLM (256M~2B)	0.25~2B	약함	지원	Apache-2.0	초경량, 설명 품질 한계

Confidential

정확도를 조금 내주고 '실행 가능성'을 사는 거래

- **양자화 - 가중치를 낮은 비트로 압축한다**
  - FP16 → 4bit (Q4_K_M = 품질/크기 균형형). 3B 모델 약 6GB → 1.8GB
  - 대가 : 미세한 정확도 손실. 대신 일반 노트북에서 돌아간다
- **GGUF - llama.cpp 진영의 단일 파일 포맷 (가중치 + 토크나이저)**
- **mmproj - 이미지를 LLM 토큰으로 바꾸는 비전 프로젝터**
  - 본체 GGUF + mmproj 두 파일이 모두 있어야 한다 - 가장 흔한 실수

Confidential

핵심은 세 줄 - 핸들러 / 모델 / 요청

- **실행 : python VLM_이미지설명.py --max-tokens 80**
- **temperature=0.0 으로 고정해야 같은 이미지에 같은 설명이 나온다**

```python
    chat_handler = Qwen25VLChatHandler(
        clip_model_path=str(VLM_MMPROJ_PATH), verbose=False
    )
    return Llama(
        model_path=str(VLM_GGUF_PATH),
        chat_handler=chat_handler,
        n_ctx=VLM_CONTEXT_SIZE,
        n_threads=os.cpu_count(),
        verbose=False,
    )

# ...

def describe(model, image: Image.Image, prompt: str, max_tokens: int) -> str:
    response = model.create_chat_completion(
        messages=[{
            "role": "user",
            "content": [
                {"type": "image_url", "image_url": {"url": to_data_url(image)}},
                {"type": "text", "text": prompt},
            ],
        }],
        max_tokens=max_tokens,
        temperature=0.0,   # 재현 가능한 출력을 위해 고정
    )
    content = response["choices"][0]["message"]["content"]
    return content.strip() if content else ""
```

전체 소스 : page37_VLM_이미지설명/VLM_이미지설명.py
https://gist.github.com/maketrust7/008796f2bcadff1daa08ea0595cf8647

Confidential

CPU 추론 1회가 전체 응답 시간을 지배한다

- **실측 : 모델 로딩 2.1초 + 추론 93.4초 (CPU)**
- **답 앞의 ':' 는 모델이 붙이는 불필요한 문자 - 2일차에 코드로 지운다**

```
명령 프롬프트

입력 이미지 : sample.jpg (810x1080)
모델 로딩 중... (CPU 추론이라 시간이 걸립니다)
모델 로딩    : 2.1s

프롬프트     : 이 이미지에 무엇이 보이는지 한국어로 간결하게 설명해줘.

[VLM 설명]
: 이미지에는 전기 버스가 보입니다. 버스는 "cero emisiones"라고 쓰여 있으며, 배경에는 건물과 나무가 보입니다.

추론 시간    : 93.4s

이 시간이 전체 응답 시간을 지배합니다.
→ 2일차에 YOLO와 VLM을 '병렬로' 실행하는 이유입니다.
```

Confidential

같은 이미지, 다른 프롬프트 - 출력이 얼마나 달라지는가

- **실행 : python VLM_프롬프트비교.py --limit 2 --max-tokens 60**
- **VLM에게는 '서술'만 시키고 '판단'과 '집계'는 다른 모듈에 맡긴다**

```
PROMPTS = [
    ("자유 설명", "이 이미지를 설명해줘."),
    ("간결 설명", "이 이미지에 무엇이 보이는지 한국어로 간결하게 설명해줘."),
    ("개수 나열", "보이는 물체를 개수와 함께 나열해줘."),
    ("판단 요구", "장애물이 있는지만 한국어로 짧게 답해줘."),
]

# ...

    for label, prompt in targets:
        print(f"--- [{label}] {prompt}")
        t0 = time.perf_counter()
        answer = ask(model, data_url, prompt, args.max_tokens)
```

전체 소스 : page39_VLM_프롬프트비교/VLM_프롬프트비교.py
https://gist.github.com/maketrust7/19d02f4786ce746f0e2e7344722f0e04

Confidential

프롬프트가 언어와 길이를 결정한다

- **실측 : "설명해줘" → 영어 279자 / 91.4초**
- **실측 : "한국어로 간결하게" → 한국어 70자 / 89.3초**
- **제어하지 않으면 영어로 답이 나온다 - LLM 입력으로 부적합**

명령 프롬프트

```
입력 이미지 : sample.jpg
비교할 프롬프트 : 2개 (CPU 추론이라 시간이 걸립니다)

--- [자유 설명] 이 이미지를 설명해줘.
: The image depicts a street scene in an urban area. In the foreground, there are three people
    walking on a sidewalk. The person on the left is wearing a light-colored coat and blue
    jeans, while the person in the middle is dressed in a dark coat and blue jeans. The person
    on the
    (91.4s, 279자)

--- [간결 설명] 이 이미지에 무엇이 보이는지 한국어로 간결하게 설명해줘.
: 이미지에는 전기 버스가 보입니다. 버스는 "cero emisiones"라고 쓰여 있으며, 배경에는 건물과 나무가 보입니다.
    (89.3s, 70자)

========================================================================
프롬프트            글자수    소요(s)
------------------------------------------------------------------------
자유 설명           279     91.4
간결 설명            70     89.3

정리: VLM에게는 '서술'만 시키고, '판단'과 '집계'는 다른 모듈에 맡긴다.
      프롬프트와 max_tokens를 고정해야 2일차 LLM 통합 결과가 재현된다.
```

Confidential

VLM 에게는 '서술'만 시키고 '판단'과 '집계'는 다른 모듈에 맡긴다

- **같은 이미지라도 프롬프트에 따라 무엇을 · 몇 개 · 어떤 말로 답할지가 달라진다**

특징	근거 (실측)	그래서
개수를 믿을 수 없다	사람을 'three people' 이라 하거나 아예 빠뜨림 (YOLO : person ×4)	집계는 YOLO
출력 형식이 흔들린다	프롬프트마다 언어 · 길이가 바뀜 (영어 279자 / 한국어 70자)	판단은 코드 · 규칙
느리다	1회 약 90초 (CPU) — YOLO 는 약 45 ms	제어 루프에 넣지 않는다
역할을 나누면 원인이 보인다	틀렸을 때 어느 모듈 문제인지 바로 찾는다	모듈별로 따로 검증

도전 문제 2 - 출력을 통제하는 프롬프트

다섯 번째 프롬프트(PROMPT_5)를 쓰고 실행하시오.

- **수정할 파일 : page42_VLM_프롬프트비교/VLM_프롬프트비교_도전2.py**
- **고칠 곳은 PROMPT_5 · MAX_TOKENS · ask() - 실행하면 ① ~ ④ 를 자동으로 확인한다**

출력이 만족해야 할 조건

① 한국어로만 답한다
② 한 문장, 40자 이내
③ 장애물이 있는지/없는지가 반드시 포함된다
④ 같은 이미지에 두 번 돌리면 같은 문장이 나온다

힌트

· 실습 3-2 의 ask() 와 비교해 본다
· ④는 프롬프트가 아니라 파라미터로 잡는다
· 길이는 프롬프트와 max_tokens 양쪽에서
· 실행 : python VLM_프롬프트비교_도전2.py
(같은 프롬프트 2회, 약 3분)

전체 소스 : page42_VLM_프롬프트비교/VLM_프롬프트비교_도전2.py
https://gist.github.com/maketrust7/c2d51b229243d7bfc58dc4f11cb74ddd

Confidential

④ 재현성은 지켜졌다. ①③ 은 실패 - 그래서 코드로 검사한다

- 재현성 → temperature=0.0 (두 번 같음)
- 길이 → 프롬프트 + max_tokens
- 실측 : 한국어 대신 한자 · 가나, 지시문을 되받아 쓰기도 한다 ("한국어로 설명해주세요.")
- 3B 모델에 긴 규격은 무리다
- → 2일차 실습 1-2 : 입력 축소 + 코드 검사

```
PROMPT_5 = ("이 이미지를 한국어 한 문장으로 설명해줘. 40자 이내로 쓰고, "
            "문장 끝에 '장애물 있음' 또는 '장애물 없음' 을 붙여줘.")
MAX_TOKENS = 40      # 프롬프트로 부탁한 길이에 상한을 한 번 더 건다

# ...

def ask(...):   # 실습 3-2 의 ask() 와 같고, 한 줄이 더 있다
        max_tokens=max_tokens,
        temperature=0.0,                    # ④ 같은 입력 → 같은 문장
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

데모 PoC 통합

Confidential

PoC = Proof of Concept, "이게 되긴 하는가" 를 최소 비용으로 확인

- **PoC ≠ Prototype ≠ Product - 단계마다 답하는 질문이 다르다**
  - PoC : "기술적으로 되는가" - 오늘, 서버 1개로 YOLO · VLM 결과를 받아 본다
  - Prototype : "사람이 쓸 수 있는가" - 3일차, 대시보드 · 시뮬레이터로 사용 흐름을 본다
  - Product : "실제로 운영할 수 있는가" - 이 과정 밖, 예외 처리 · 인증 · 배포까지
- **시작 전에 '성공 기준' 을 한 문장으로 적는다**
  - 오늘 : 이미지 1장으로 YOLO 결과와 VLM 설명을 함께 받을 수 있는가
- **오늘 만들 것 : FastAPI 서버 1개 + 엔드포인트 2개**
  - /api/yolo-detect, /api/vlm-describe - 입력은 이미지, 출력은 텍스트, 소요 시간 로그

Confidential

PoC 실패의 1순위 원인은 '범위 확장' 이다

- **오늘 하는 것 - 두 엔드포인트, 동시 호출 비교, 소요 시간 로그**
- **오늘 하지 않는 것 - 언제 할지만 정해 둔다**
  - LLM 통합 · 웹캠 화면 → 2일차, 의도 파악 · 도구 호출 → 3일차, 예외 처리 · 인증 → 범위 밖
- **판단 기준 : 새 기능이 떠오르면 먼저 이렇게 묻는다**
  - "이 기능이 PoC 의 성공 기준을 확인하는 데 꼭 필요한가?"
  - 예) 오늘의 성공 기준 : 이미지 1장으로 YOLO 결과 + VLM 설명을 함께 받는다
  - 필요하면 만든다. 아니면 위 '언제 할지' 목록에 적고 넘어간다

Confidential

두 엔드포인트 모두 '이미지 in → 텍스트 out' 으로 통일

- 서버 실행 : python PoC_FastAPI서버.py (창은 켜 둔다)
- 또는 : uvicorn PoC_FastAPI서버:app --port 8000
- 인터페이스를 통일해야 2일차 통합이 쉬워진다
- /api/health (브라우저 주소창) 로 지연 로딩 상태 확인
- 서버만 띄우면 yolo_loaded · vlm_loaded 는 false
  기다려도 그대로
- 첫 요청이 들어와야 모델을 올리고 true 가 된다
  (49쪽 클라이언트로 호출)

```python
@app.post("/api/yolo-detect")
async def yolo_detect(request: Request):
    try:
        image = await _read_image(request)
    except (ValueError, UnidentifiedImageError) as exc:
        return JSONResponse({"error": str(exc)}, status_code=400)

    t0 = time.perf_counter()
    summary = await run_in_threadpool(_run_yolo_inference, image)
    elapsed_ms = round((time.perf_counter() - t0) * 1000)
    print(f"[yolo] {elapsed_ms}ms  {summary}")
    return {"summary": summary, "elapsed_ms": elapsed_ms}

# ...

@app.post("/api/vlm-describe")
async def vlm_describe(request: Request):
    try:
        image = await _read_image(request)
    except (ValueError, UnidentifiedImageError) as exc:
        return JSONResponse({"error": str(exc)}, status_code=400)

    t0 = time.perf_counter()
    description = await run_in_threadpool(_run_vlm_inference, image)
    elapsed_ms = round((time.perf_counter() - t0) * 1000)
    print(f"[vlm ] {elapsed_ms}ms  {description[:40]}...")
    return {"description": description, "elapsed_ms": elapsed_ms}
```

전체 소스 : page47_PoC_서버/PoC_FastAPI서버.py
https://gist.github.com/maketrust7/6e3f29eb1cda5ad56c35f4792aeabea5

Confidential

모델 추론은 동기(블로킹) 작업이다

- **위임하지 않으면 서버 전체가 멈춘다**
  - VLM 1회 약 92초 동안 다른 모든 요청이 대기한다 → run_in_threadpool() 로 위임
  - async def 라우트라고 FastAPI 가 자동으로 스레드풀에 넘겨주지 않는다
- **모델 로딩은 지연(lazy) + 락(lock) - Object Detection 실습에서 검증한 그 패턴**
- **추론 호출도 한 번에 하나 - llama.cpp 인스턴스는 동시 호출에 안전하지 않다**
  - 2일차 대시보드 서버는 추론도 락으로 감싼다

Confidential

터미널 2개 - 서버와 클라이언트

- **터미널 1 : page47_PoC_서버 에서 서버 실행**
- **터미널 2 : page49_PoC_클라이언트 에서 --parallel 호출**

```python
def post_image(base_url: str, endpoint: str, image_bytes: bytes,
               timeout: float) -> dict:
    url = f"{base_url.rstrip('/')}{endpoint}"
    response = httpx.post(
        url, content=image_bytes,
        headers={"Content-Type": "image/jpeg"}, timeout=timeout,
    )
    response.raise_for_status()
    return response.json()

# ...

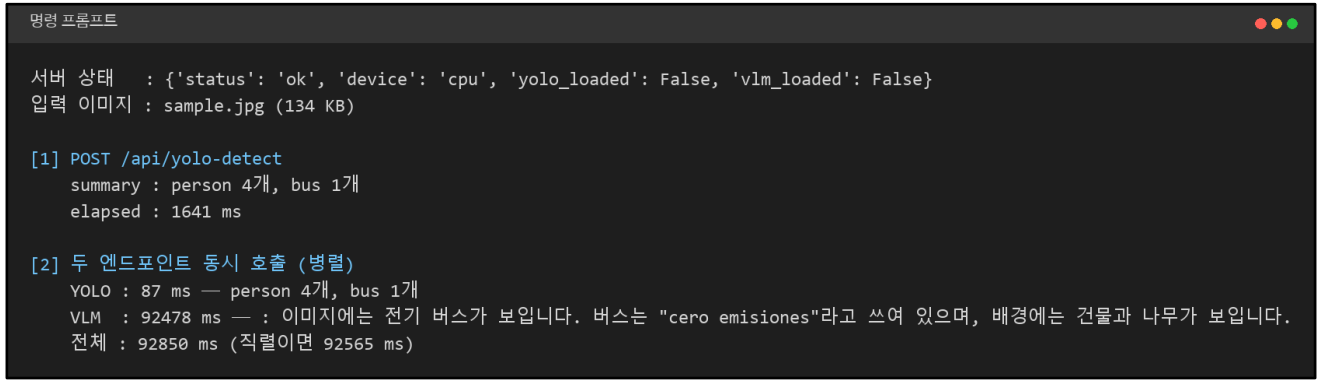
        with ThreadPoolExecutor(max_workers=2) as pool:
            f_yolo = pool.submit(post_image, args.url, "/api/yolo-detect",
                                 image_bytes, args.timeout)
            f_vlm = pool.submit(post_image, args.url, "/api/vlm-describe",
                                image_bytes, args.timeout)
            yolo, vlm = f_yolo.result(), f_vlm.result()
```

전체 소스 : page49_PoC_클라이언트/PoC_클라이언트_호출테스트.py
https://gist.github.com/maketrust7/a435896d9c6548735f5c3e5e1096e23b

Confidential

YOLO와 VLM의 시간 차가 그대로 드러난다

- **yolo_loaded:false → 서버는 떴지만 모델은 아직 안 올라왔다 (지연 로딩의 증거)**
- **첫 호출 1,641 ms → 두 번째부터 87 ms (모델 로딩 차이)**
- **실측 : YOLO 87 ms vs VLM 92,478 ms**



Confidential

측정값이 다음 차시의 설계를 결정한다

- **시간 - YOLO 87 ms / VLM 92,478 ms (실측, 약 1,060배)**
  - 가장 느린 경로가 곧 전체 응답 시간 → VLM 단축이 2일차 과제
- **품질 - 모델이 무엇을 틀리는지 기록한다**
  - 실측 : VLM 은 "three people", YOLO 는 person 4 - 개수는 YOLO 를 믿는다 (2일차 규칙)
- **인터페이스 - 출력 텍스트 형식을 고정해 LLM 입력 템플릿으로 굳힌다**

도전 문제 3 - 병렬로 부르면 얼마나 빨라질까

--parallel 을 실행하기 전에 예측하고, 측정해 비교하시오.

먼저 종이에 적고 시작하세요

Q1. 순차 호출의 총 소요 시간은?

Q2. 병렬로 바꾸면 총 소요 시간은?

Q3. 몇 % 빨라지는가?

```
        with ThreadPoolExecutor(max_workers=2) as pool:
            f_yolo = pool.submit(post_image, args.url, "/api/yolo-detect",
                                 image_bytes, args.timeout)
            f_vlm = pool.submit(post_image, args.url, "/api/vlm-describe",
                                image_bytes, args.timeout)
            yolo, vlm = f_yolo.result(), f_vlm.result()
```

실행 (--parallel 이 위 코드)

python PoC_클라이언트_호출테스트.py --parallel

(서버는 터미널 1에서 계속 띄워 둔 상태)

Confidential

병렬의 이득은 '두 번째로 느린 작업' 만큼 - 여기서는 87 ms 뿐

- **전체 시간의 99.9% 가 VLM 한 곳에 몰려 있다 = 단일 병목**
- **+285 ms 는 HTTP 전송 · 측정 방식 차이 수준 - 사실상 이득 0**
- **할 일은 병렬화가 아니라 VLM 단축 (입력 이미지 축소 · 출력 줄이기 · GPU) - 2일차**

	계산	총 소요 시간	개선
순차 (서버 측 추론 합)	87 ms + 92,478 ms	92,565 ms	—
병렬 (클라이언트 측정)	max(87, 92,478) + 전송	92,850 ms	0 %

Object Detection Finetuning

Confidential

성능 올리기가 아니라 '우리 환경에 맞추기' 다

- **① 클래스 부재 - COCO 80개 클래스에 찾는 물체가 없다**
  - 예 : 특정 장애물, 주행 표식. 임계값을 내려도 없는 클래스는 나오지 않는다
- **② 도메인 갭 - 라즈베리파이 카메라 · 조명 · 각도는 인터넷 사진과 다르다**
- **③ 작은 모델 + 내 데이터 - 좁은 도메인에서는 큰 범용 모델을 이긴다**

Confidential

라벨 값은 모두 0~1 로 정규화된다

- **수집 : 실제 운용과 같은 조건으로 촬영 (같은 카메라 · 각도 · 조명)**
- **클래스당 100~200장 권장 / 실습은 40장으로 흐름만 체험**
- **도구 : Roboflow / labelImg / CVAT**
- **생성 위치 : 2회차_day1₩dataset₩**

```
# labels/train/img001.txt  — 값은 모두 0~1 정규화
# class_id  x_center  y_center  width  height
0  0.512  0.437  0.180  0.260
1  0.218  0.690  0.095  0.140

# dataset/data.yaml  (스크립트가 만든다)
path: <프로젝트>/source/2회차_day1/dataset
train: images/train
val: images/val
names:
  0: obstacle
  1: marker
```

전체 소스 : page56_데이터셋_준비/데이터셋_준비와검증.py
https://gist.github.com/maketrust7/4f1c35d95a0d4eca50e624f8d3307b87

Confidential

라벨 범위 · 짝 누락 · 클래스 분포까지 검증한다

- **실행 : python 데이터셋_준비와검증.py --demo**
- **실측 : train 40장 / val 10장**

명령 프롬프트

```
데모 데이터셋 생성 완료: train 40장 / val 10장
data.yaml 작성: C:\work\vlm9\source\2회차_day1\dataset\data.yaml

[train] 이미지 40장

[val] 이미지 10장

[클래스 분포]
  obstacle      38개
  marker        36개

[PASS] 데이터셋 검증 통과
  (0s)
```

Confidential

반드시 사전학습 가중치에서 출발한다 (전이학습)

- **처음부터 학습(scratch)하면 수만 장이 필요하다**
- **실행 : python YOLO_파인튜닝_학습.py --epochs 10**
- **CPU 에서 약 30초 (데모 40장, imgsz 320)**

```python
model = YOLO(str(PRETRAINED) if PRETRAINED.exists() else "yolo11n.pt")

t0 = time.perf_counter()
model.train(
    data=str(data_path),
    epochs=args.epochs,
    imgsz=args.imgsz,
    batch=args.batch,
    patience=args.patience,
    device=device,
    project=str(RUNS_DIR),
    name=args.name,
    exist_ok=True,
    verbose=False,
    plots=False,
)
```

전체 소스 : page58_파인튜닝_학습/YOLO_파인튜닝_학습.py
https://gist.github.com/maketrust7/45af1ae393d131ec65adb582107462aa

Confidential

epoch 마다 val mAP 가 어떻게 움직이는지 본다

- **box_loss / cls_loss 가 내려가는지 확인**
- **실측 : val mAP50 0.354 → 0.968**
- **수치는 환경마다 다르다 (학습의 무작위성)**

명령 프롬프트

```
Starting training for 10 epochs...

  Epoch    box_loss  cls_loss  dfl_loss  |    P       R     mAP50  mAP50-95
  ------------------------------------------------------------------------
    1/10    0.8560    3.2480    1.0000  |  0.0062  1.0000  0.3540   0.2990
    2/10    0.6118    2.7150    0.8867  |  0.0066  1.0000  0.5410   0.4050
    3/10    0.6307    2.4040    0.8710  |  0.0066  1.0000  0.5410   0.4050
    4/10    0.7572    1.9800    0.9675  |  0.0077  1.0000  0.6540   0.4090
    ...   (중간 5 epoch 생략)
   10/10    0.5059    1.1380    0.8563  |  0.1080  1.0000  0.9680   0.8400
```

Confidential

한 번에 여러 값을 바꾸지 않는다

- **epochs - 적으면 미학습, 많으면 과적합. patience 로 조기 종료**
  - 실측 : 10 epochs 에서 mAP50 0.354 → 0.968 - 그래도 확신은 낮다 (도전 4)
- **imgsz - 운용 해상도와 맞춘다. 키우면 정확도↑ 속도↓**
- **batch - OOM(메모리 부족) 이 나면 가장 먼저 줄인다. 16 → 8 → 4**
- **lr0 · augment - 오늘은 기본값. 하나씩 바꾸고 mAP 를 기록한다**

Confidential

같은 val 셋으로 두 번 평가해야 비교가 성립한다

- **실행 : python YOLO_파인튜닝_평가비교.py**
- **max_conf : 모델이 낸 가장 높은 확신도**
- **--conf 0.25 : 실제 임계값으로 재평가**

```
    metrics = model.val(data=str(data_path), imgsz=imgsz, device=device, conf=conf,
                        verbose=False, plots=False)

# ...

    max_conf = max((float(r.boxes.conf.max()) for r in
                    model.predict([str(p) for p in val_images], imgsz=imgsz, conf=0.001,
                                  device=device, verbose=False) if len(r.boxes)),
                   default=0.0)
```

전체 소스 : page61_파인튜닝_평가/YOLO_파인튜닝_평가비교.py
https://gist.github.com/maketrust7/72a21450f8e2934541279eb4ccb984fb

Confidential

이 표에는 이상한 숫자가 하나 숨어 있다

- **before 0 : 사전학습 모델은 obstacle · marker 를 모른다 → 맞힌 박스 0개**
- **mAP50 0.968 : 정답 박스를 거의 항상 1순위로 찾는다**
- **recall 1.0 : 정답 16개를 하나도 놓치지 않았다**
- **precision 0.106 : 평가 conf 0.001 이라 헛검출이 섞여 낮다**
- **max_conf 0.042 : 가장 자신 있는 답도 4%**
- **실제로 쓸 때는 conf 0.25 미만 박스를 버린다 → 전부 버려져 장애물을 하나도 못 찾는다 → 도전 4**

```
명령 프롬프트

[before] 사전학습 모델 평가 중...
평가 임계값 conf = 0.001
YOLO11n summary (fused): 100 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs

                   all         10         16          0          0          0          0
[after ] 파인튜닝 모델 평가 중...
YOLO11n summary (fused): 101 layers, 2,582,542 parameters, 0 gradients, 6.3 GFLOPs

                   all         10         16      0.106          1      0.968       0.84

stage |    weights |      mAP50 |   mAP50_95 |  precision |     recall |   max_conf |   infer_ms
------------------------------------------------------------------------------------------------
    --------
before | yolo11n.pt |        0.0 |        0.0 |        0.0 |        0.0 |     0.3793 |
        20.3
after |    best.pt |     0.9683 |       0.84 |     0.1056 |        1.0 |     0.0418 |       20.1

mAP50 변화    : 0.0000 -> 0.9683 (+0.9683)
mAP50-95 변화 : 0.0000 -> 0.8400 (+0.8400)

사전학습 모델은 우리 클래스를 모르기 때문에 점수가 거의 0이다.
이 차이가 '도메인 정렬(domain alignment)'의 효과다.

[주의] 파인튜닝 모델의 최고 확신도 0.0418 < 0.25
  순위(mAP)는 맞지만 확신이 낮다 — 실제로 쓰는 conf 0.25 에서는 아무것도 검출하지 못한다.
  확인 : python YOLO_파인튜닝_평가비교.py --conf 0.25

[과적합 징후 체크]
  - train loss는 내려가는데 val mAP가 정체/하락 -> 데이터 부족 또는 epochs 과다
  - 학습 데이터에서만 잘 맞고 새 사진에서 급락 -> 다양성 부족

CSV 저장: C:\work\vlm9\source\2회차_day1\page61_파인튜닝_평가\finetune_compare.csv
  (5s)
```

도전 문제 4 - 이 모델, 써도 될까

Confidential

stage	mAP50	mAP50-95	precision	recall	max_conf
before (yolo11n.pt)	0.0000	0.0000	0.0000	0.0000	0.3793
after (best.pt)	0.9683	0.8400	0.1056	1.0000	0.0418

세 가지를 답하시오

- **Q1. mAP50 0.968 인데 max_conf 는 0.042 다. 이 모델은 어떤 상태인가?**
- **Q2. 로봇에 올려 conf 0.25 로 쓰면 어떤 일이 벌어지는가?**
- **Q3. 다음에 할 조치를 2가지 쓰시오.**

도전 4 해설 - 순위는 맞지만 확신이 없다

Confidential

mAP 는 줄 세우기, conf 는 확신의 크기 - 둘은 다른 이야기다

- **A1. 과소학습 - 정답 박스가 항상 앞 순위(mAP↑)지만 점수가 0.04 이하로 낮다**
- **A2. conf 0.25 에서는 아무것도 검출하지 못한다 - 장애물을 못 보고 부딪힌다 (아래 실측)**
- **A3. ① 학습을 더 (epochs · 데이터 · 배경 이미지) ② 실제로 쓸 conf 0.25 로 다시 평가해 precision(정밀도) · recall(재현율) 을 함께 본다**

python YOLO_파인튜닝_평가비교.py --conf 0.25 (실측)

```
[before] 사전학습 모델 평가 중...
평가 임계값 conf = 0.25
YOLO11n summary (fused): 100 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs

                   all         10         16          0          0          0          0
[after ] 파인튜닝 모델 평가 중...
YOLO11n summary (fused): 101 layers, 2,582,542 parameters, 0 gradients, 6.3 GFLOPs

                   all         10         16          0          0          0          0

     stage |    weights |      mAP50 |   mAP50_95 |  precision |     recall |   max_conf |   infer_ms
-----------------------------------------------------------------------------------------------------
    before | yolo11n.pt |        0.0 |        0.0 |        0.0 |        0.0 |     0.3793 |       20.0
     after |    best.pt |        0.0 |        0.0 |        0.0 |        0.0 |     0.0418 |       19.7
    ... (이하 생략)
```

Confidential

내용 정리

- ① 단일 모델이 아니라 역할 분담 - 개수는 Object Detection, 맥락은 VLM, 판단은 LLM
- ② 모델 선정은 취향이 아니라 기준 - 정확도 · 속도 · 메모리 · 라이선스 + 이식 계획
- ③ 인식 결과는 모두 '텍스트'로 통일 - 멀티모달 통합의 실질적 인터페이스
- ④ 추론은 블로킹 - 스레드풀 위임, 지연 로딩 + 락은 선택이 아니라 필수
- ⑤ 최적화 전에 측정 - 병목이 하나면 병렬화는 1초도 못 줄인다 (도전 3)
- ⑥ 지표는 읽는 것 - mAP 가 높아도 확신(conf)이 낮으면 못 쓴다 (도전 4)

Confidential

필수 및 선택 기능 구현

- **① (필수) 성능 기록표 완성 - 내 PC의 measurement.csv**
  - 참고 : 28~29쪽 (실습 2-2)
- **② (필수) VLM 프롬프트 비교 메모 1장 - 어떤 프롬프트가 LLM 입력에 맞는가**
  - 참고 : 39~41쪽 (실습 3-2)
- **③ (선택) --epochs 30 으로 다시 학습해 max_conf 가 0.25 를 넘는지 본다**
  - 참고 : 58~59쪽 (실습 5)
- **④ (선택) 도전 4 를 내 PC에서 확인한다**
  - python YOLO_파인튜닝_평가비교.py --conf 0.25 - max_conf 와 precision(정밀도) · recall(재현율)을 기록한다
  - 참고 : 61~64쪽 (평가 · 도전 4)

Confidential

오늘 만든 YOLO · VLM 엔드포인트를 웹 대시보드에 통합한다

- **인식 결과를 LLM 이 읽을 Text 규격으로 고정한다**
  - 오늘 본 '좌표 → 한 문장' 변환 (YOLO), 한 문장 + 장애물 유무 (VLM, 도전 2)
- **세 번째 입력 - MediaPipe 손 제스처**
- **웹 대시보드 서버 - 제공되는 화면(웹캠)의 요청을 처리하는 API 를 만든다**
  - /api/yolo-detect · /api/vlm-describe 는 오늘 그대로, VLM 은 입력을 줄여 빠르게 (도전 3 의 결론)
- **LLM(Qwen3-4B) 이 세 가지 Text 로 상황을 설명한다 - 개수는 YOLO 를 따른다**
- **준비물 : 오늘 환경 + mediapipe (2일차 첫 시간에 설치) + 웹캠 (제공)**

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Q & A

Confidential

모든 명령은 C:₩work₩core_pjt₩source₩2회차_day1 에서 실행한다

- 옵션은 -h 로 확인할 수 있다
- 생성물은 각 실습 폴더에 생긴다
- measurement.csv / finetune_compare.csv
- dataset/ · runs/ 는 공용 산출물

```
# 실습 한 건 = 폴더 한 개. 폴더 앞 숫자 = 이 강의자료 쪽번호
cd C:\work\core_pjt\source\2회차_day1

# Part 1 — 환경 확인
cd page15_환경체크          && python 환경체크_설치확인.py

# Part 2 — Object Detection
cd ..\page23_모델로딩_지연과락 && python YOLO모델_지연로딩과락.py
cd ..\page25_YOLO_첫추론       && python YOLO_첫추론.py --sweep
cd ..\page28_YOLO_성능측정     && python YOLO_성능측정_기록표.py --runs 10

# Part 3 — VLM (CPU 1회 약 90초)
cd ..\page37_VLM_이미지설명     && python VLM_이미지설명.py --max-tokens 80
cd ..\page39_VLM_프롬프트비교   && python VLM_프롬프트비교.py --limit 2 --max-tokens 60

# Part 4 — PoC 통합 (터미널 2개)
cd ..\page47_PoC_서버          && python PoC_FastAPI서버.py
cd ..\page49_PoC_클라이언트     && python PoC_클라이언트_호출테스트.py --parallel

# Part 5 — 파인튜닝 (학습 약 30초)
cd ..\page56_데이터셋_준비      && python 데이터셋_준비와검증.py --demo
cd ..\page58_파인튜닝_학습      && python YOLO_파인튜닝_학습.py --epochs 10
cd ..\page61_파인튜닝_평가      && python YOLO_파인튜닝_평가비교.py
```

Confidential

실습 중 가장 많이 나오는 질문

- **먼저 이 표에서 찾아본 뒤 질문해 주세요**

증상 / 에러 메시지	원인	해결
ModuleNotFoundError: llama_cpp	VSCode가 .venv를 인터프리터로 잡지 않음	Ctrl+Shift+P → Python: Select Interpreter → .venv 선택 후 터미널 재시작
VLM 본체 / mmproj 파일이 없습니다	가중치 미다운로드	models₩ 에 GGUF 2개를 모두 배치
image slice encoded in ... 로그	llama.cpp 진행 로그	오류가 아님. 무시
huggingface-cli ... no longer works	옛 다운로드 명령	pip install huggingface_hub 후 hf download
activate 시 '스크립트를 실행할 수 없으므로'	PowerShell 실행 정책	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
UnicodeEncodeError: cp949	콘솔이 표현 못 하는 문자 출력	출력 문자열을 cp949 안전 문자로 교체
[Errno 10048] bind on address	8000 포트를 이미 다른 서버가 사용	기존 서버 종료 또는 --port 8001
torch.cuda.is_available() False	torch CPU 빌드	CUDA 빌드로 재설치 (RTX 50 시리즈는 cu128+)
CUDA out of memory	batch/imgsz 과다	batch 16 → 8 → 4 로 축소

Confidential

PoC 서버가 주고받는 형식 - 이미지 in → 텍스트 out

- **page47 PoC_FastAPI서버.py - 같은 경로를 2일차 대시보드 서버 · 3일차 API 설계가 이어 쓴다**

메서드 · 경로	요청	응답 (200)	오류
GET /api/health	—	{status, device, yolo_loaded, vlm_loaded}	—
POST /api/yolo-detect	본문 = 이미지 바이트	{summary: 'person 3개, bus 1개', elapsed_ms}	400 {error: 메시지}
POST /api/vlm-describe	본문 = 이미지 바이트	{description: 문자열, elapsed_ms}	400 {error: 메시지}

2일차 · 3일차에서 바뀌는 것	내용
yolo-detect 응답	summary 문자열 → objects 목록 [{label, confidence, box}] (화면이 박스를 그린다)
오류 응답	{error: 메시지} → {error: 코드} (empty_image · invalid_image · yolo_failed …)
새 경로	2일차 : GET / · /mediapipe/... · POST /api/llm-answer, 3일차 : llm-answer 가 도구 호출로

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

내일 방송에서 만나요!

삼성청년SW·AI아카데미